

CUSTOMER SUPPORT TICKET API

End-to-end DevOps project: CI/CD on Kubernetes

Python / Flask • Git & GitHub • Docker • Jenkins • Kubernetes (Minikube)

SOURCE

BUILD

DEPLOY

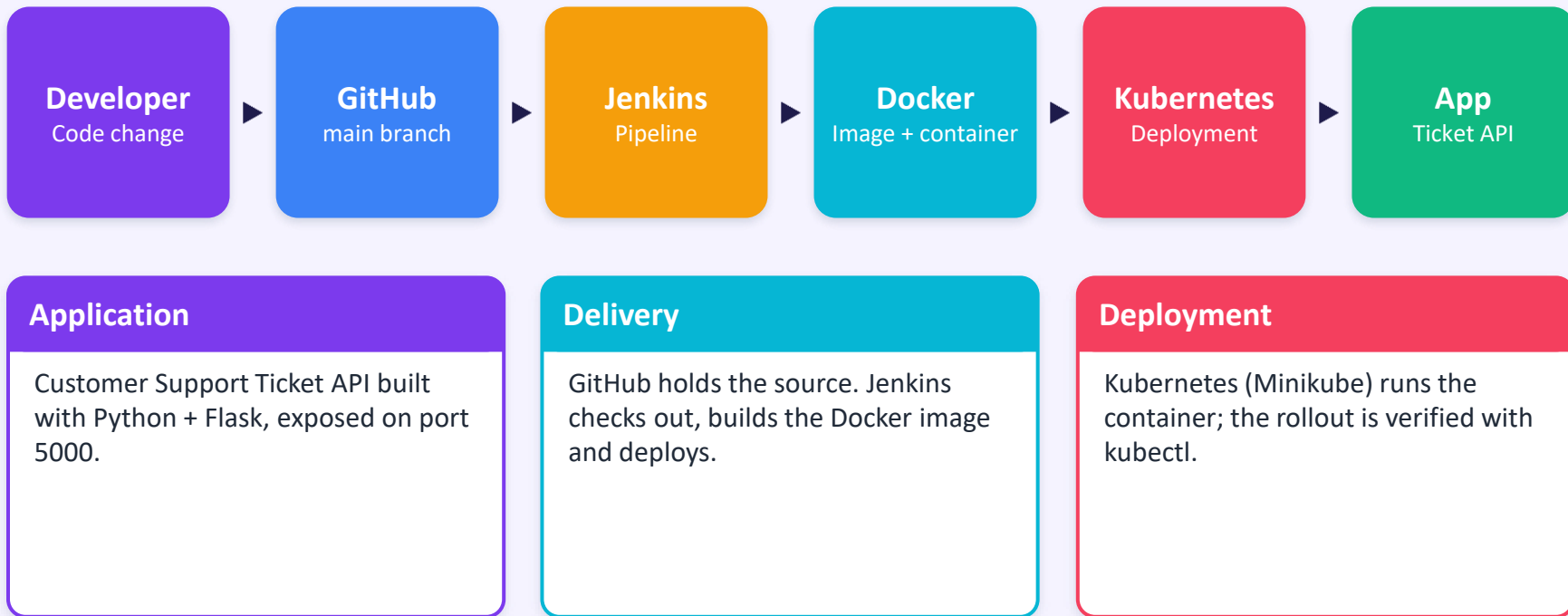
VERIFY

Rahul Bhatt | BCA Student | Aspiring Cloud Engineer

Project Overview & Architecture

2

One small app, taken through a real DevOps delivery workflow



Source Control: GitHub

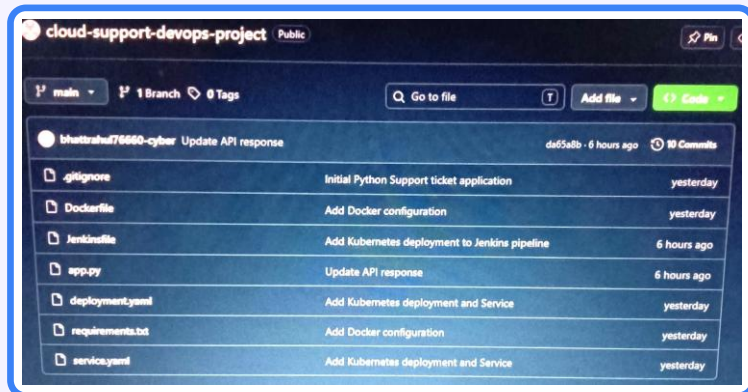
3

GitHub is the source of truth for the pipeline

Repository

- cloud-support-devops-project (public)
- Branch: main • 10 commits
- app.py, Dockerfile, Jenkinsfile, deployment.yaml, service.yaml
- Flow: add → commit → push → Jenkins

Jenkins pulls the code from this repo; there is no local copy involved in the pipeline.



GitHub repo: files + commit history

```
(venv) PS C:\DEVOPS-PROJECT> git push -u origin main
Delta compression using up to 12 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (4/4), 607 bytes | 607.00 KiB/s, done.
Total 4 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/bhatttrahul76660-cyber/cloud-support-devops-project.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
(venv) PS C:\DEVOPS-PROJECT>
```

git push -u origin main from VS Code terminal

Application Layer

4

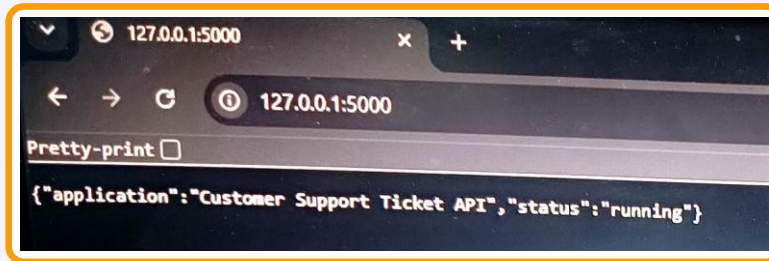
A simple Flask API, so the focus stays on DevOps

app.py

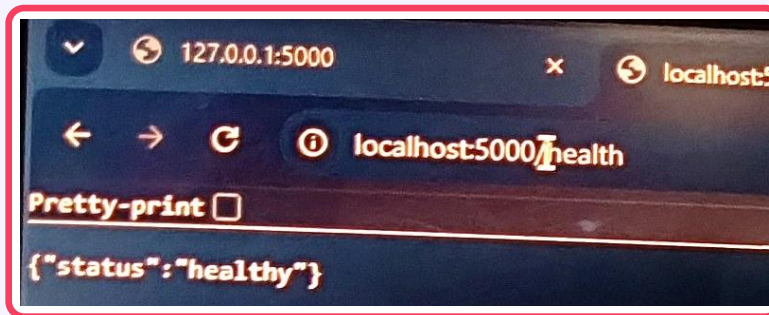
```
from flask import Flask, jsonify
app = Flask(__name__)

@app.route("/")
def home():
    return jsonify({
        "application": "Customer Support Ticket API is running",
        "status": "running"
    })

@app.route("/health")
def health():
    return jsonify({"status": "healthy"})
```



GET / → application status

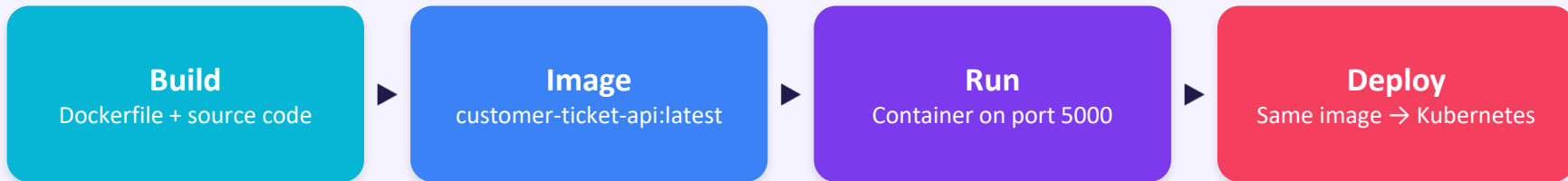


GET /health → {"status": "healthy"}

Containerization: Docker

5

Package the app once, run it the same way everywhere



	Name	Container ID	Image	Port(s)	CPU (%)	Last	Actions
☐	customer-ticket	14e601c6be2f	customer-t	5000:5000	0%	6 ho	
☐	Jenkins	09096b477290	custom-jen	50000:50000 Show all ports (2)	1.09%	24 s	
☐	react-container	98f54128cd92	my-react-a	3000:3000	0%	8 da	
☐	minikube	90ccce58260d	k8s-miniku		0%	7 ho	

Docker Desktop: customer-ticket-api (5000:5000), Jenkins and minikube containers

KEY COMMANDS

```
docker build -t  
customer-ticket-api:latest .
```

```
docker run -p 5000:5000  
customer-ticket-api:latest
```

*Repeatable runtime; the image is the artifact
Jenkins hands to Kubernetes.*

CI/CD: Jenkins Pipeline

Declarative pipeline from the Jenkinsfile in GitHub

6

1 Checkout
Fetch from GitHub

2 Build
Docker image

3 Run
Start container

4 Deploy
Update deployment

5 Rollout
Verify status

```
Started by user admin
Obtained Jenkinsfile from git https://github.com/bhattrahul76660-cyber/cloud-support-devops-project.git
[Pipeline] Start of Pipeline
[Pipeline] node
Running on Jenkins in /var/jenkins_home/workspace/customer-ticket-api-pipeline
[Pipeline] {
[Pipeline] stage
[Pipeline] { (Declarative: Checkout SCM)
[Pipeline] checkout
Selected Git installation does not exist. Using Default
The recommended git tool is: NONE
No credentials specified
> /usr/bin/git rev-parse --resolve-git-dir /var/jenkins_home/workspace/customer-ticket-api-pipeline/.git # timeout=10
Fetching changes from the remote Git repository
> /usr/bin/git config remote.origin.url https://github.com/bhattrahul76660-cyber/cloud-support-devops-project.git
timeout=10
Cleaning workspace
> /usr/bin/git rev-parse --verify HEAD # timeout=10
Resetting working tree
> /usr/bin/git reset --hard # timeout=10
> /usr/bin/git clean -ffdx # timeout=10
Fetching upstream changes from https://github.com/bhattrahul76660-cyber/cloud-support-devops-project.git
> /usr/bin/git --version # timeout=10
> git --version # 'git version 2.47.3'
> /usr/bin/git fetch --tags --force --prune --depth=1 https://github.com/bhattrahul76660-cyber/cloud-support-devops-project.git # timeout=10
```

Console: Jenkinsfile obtained from GitHub, checkout SCM

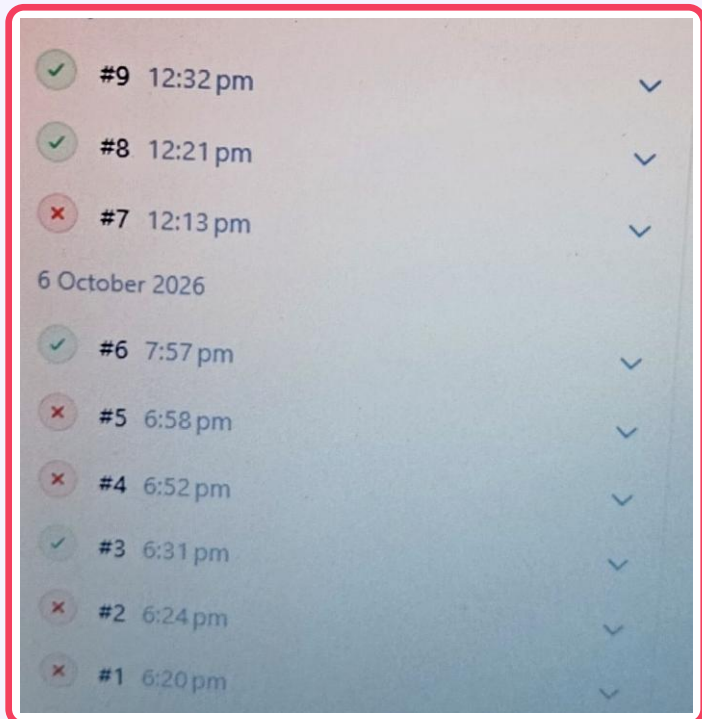
```
+ docker run -d --name customer-ticket-api -p 5000:5000 customer-ticket-api:latest
14e601c6ba2ff1ec999bfc51cf59b0c4105d3f3c1b20617a08acf9b0626e8bd8
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Deploy to Kubernetes)
[Pipeline] sh
+ export KUBECONFIG=/tmp/kubeconfig
+ kubectl set image deployment/customer-ticket-api customer-ticket-api=customer-ticket-api:latest
+ kubectl rollout status deployment/customer-ticket-api
deployment "customer-ticket-api" successfully rolled out
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```

Console: kubectl rollout → "successfully rolled out", Finished: SUCCESS

Jenkins Build History: Fail → Fix → Success

7

9 builds, 5 failed, 4 successful — each failure was debugged



5 failed

Builds #1, #2, #4, #5, #7 failed during Git tool, checkout and pipeline-syntax setup.

4 successful

Builds #3, #6, #8, #9 passed; #9 ran Checkout → Build → Run → Deploy → Rollout.

What it shows

Real debugging: read console logs, fix config, rebuild until the pipeline is green.

Orchestration: Kubernetes (Minikube)

8

Real cluster output

```
minikube v1.39.0 on Microsoft Windows 11 Home Single Language 26H2
Using the docker driver based on existing profile
Starting "minikube" primary control-plane node in "minikube" cluster
Pulling base image v0.0.51 ...
Restarting existing docker container for "minikube" ...
Preparing Kubernetes v1.37.0 on contained 2.3.4 ...
Verifying Kubernetes components...
  - Using image docker.io/kubernetes/dashboard:v2.7.0
  - Using image gcr.io/k8s-minikube/storage-provisioner:v5
  - Using image docker.io/kubernetes/metrics-scrafer:v1.0.8
Failing to connect to https://registry.k8s.io/ from both inside the minikube container and host machine
To pull new external images, you may need to configure a proxy: https://minikube.sigs.k8s.io/docs/reference/networking/p
roxy/
Some dashboard features require the metrics-server addon. To enable all features please run:

  minikube addons enable metrics-server

Enabled addons: storage-provisioner, dashboard, default-storageclass
Done! kubectl is now configured to use "minikube" cluster and "default" namespace by default
(venv) PS C:\DEVOPS-PROJECT>
```

```
(venv) PS C:\DEVOPS-PROJECT> kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
customer-ticket-api-644955bdf-4rkp2  1/1     Running   1 (88s ago)  22h
customer-ticket-api-644955bdf-wmlwh  1/1     Running   1 (88s ago)  22h
(venv) PS C:\DEVOPS-PROJECT> minikube service list
```

NAMESPACE	NAME	TARGET PORT	URL
default	customer-ticket-service	5000	
default	kubernetes	No node port	
kube-system	kube-dns	No node port	
kubernetes-dashboard	dashboard-metrics-scraper	No node port	
kubernetes-dashboard	kubernetes-dashboard	No node port	

```
(venv) PS C:\DEVOPS-PROJECT>
```

kubectl get pods → 2/2 Running; minikube service list

minikube start → kubectl configured to use "minikube"

Deployment

customer-ticket-api (2 replicas). Jenkins updates the image with kubectl set image, then checks the rollout.

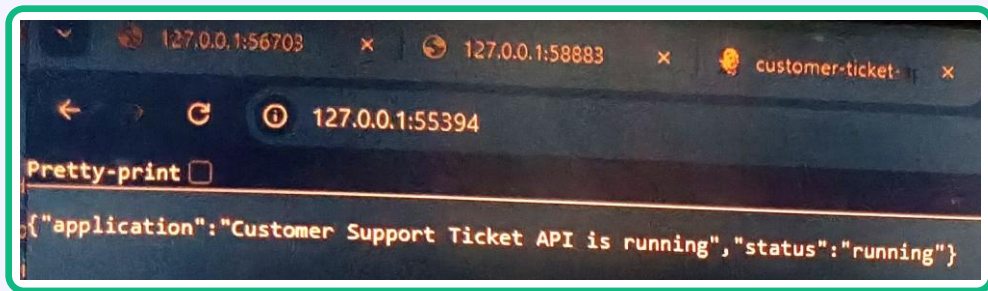
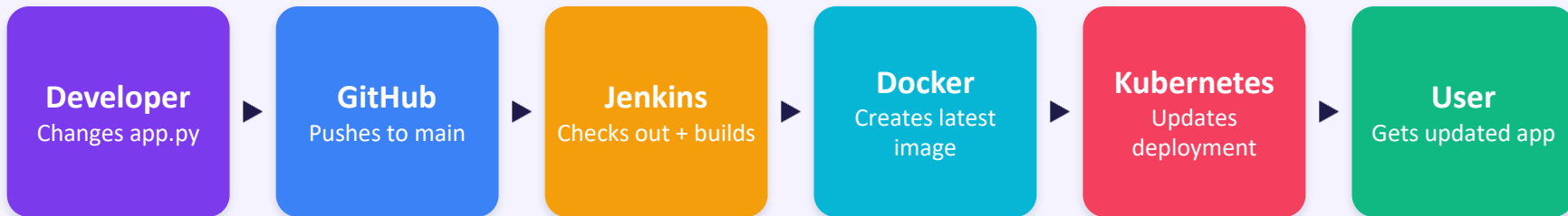
Key concept

Jenkins automates the change; Kubernetes runs and manages the containers.

End-to-End Result

9

A code change reached the running cluster



Updated response served from the cluster via a minikube service URL

Result

The developer only changes code; the pipeline handles repeatable delivery; Kubernetes runs the workload.

Troubleshooting: Jenkins & Git

10

Setup errors that were diagnosed and fixed

Area	Observed issue	Fix / learning
Jenkins	Selected Git installation does not exist	Fix Git/tool configuration, use the available installation
Jenkins → GitHub	Fetching remote repo / max checkout retry	Check repo URL + credentials, retry checkout
Repository	Empty Git repo at first setup	Add files, commit, push to main
Jenkinsfile	Pipeline syntax / shell formatting issue	Valid Declarative syntax; shell commands in sh steps
Pipeline runs	5 of first 9 builds failed	Read console logs, fix, rebuild until #9 green

Troubleshooting: Docker & Kubernetes

11

Real errors from this project, with the fix

```
(venv) PS C:\DEVOPS-PROJECT> minikube status
E1007 16:50:10.767613 27692 status.go:178] status error: host: state: unknown state "minikube": docker container inspect #
minikube --format={{.State.Status}}: exit status 1
stdout:

stderr:
failed to connect to the docker API at npipe:////./pipe/dockerDesktopLinuxEngine; check if the path is correct and if the da
emon is running: open //.pipe/dockerDesktopLinuxEngine: The system cannot find the file specified.
E1007 16:50:10.767613 27692 status.go:127] status error: host: state: unknown state "minikube": docker container inspect #
minikube --format={{.State.Status}}: exit status 1
stdout:

stderr:
failed to connect to the docker API at npipe:////./pipe/dockerDesktopLinuxEngine; check if the path is correct and if the da
emon is running: open //.pipe/dockerDesktopLinuxEngine: The system cannot find the file specified.
o (venv) PS C:\DEVOPS-PROJECT>
```

minikube status: Docker API not reachable

```
(venv) PS C:\DEVOPS-PROJECT> kubectl config current-context
minikube
(venv) PS C:\DEVOPS-PROJECT> kubectl apply -f deployment.yaml
error: error validating "deployment.yaml": error validating data: failed to download openapi: Get "https://127.0.0.1:52683/openapi/v2?time
out=3s": dial tcp 127.0.0.1:52683: connect: No connection could be made because the target machine actively refused it.; if you choose
to ignore these errors, turn validation off with --validate=false
o (venv) PS C:\DEVOPS-PROJECT>
```

kubectl apply: connection refused to API server

```
(venv) PS C:\DEVOPS-PROJECT> kubectl get pods
```

NAME	READY	STATUS	RESTARTS	AGE
customer-ticket-api-74c585b587-sbfp7	0/1	ImagePullBackOff	0	108s
customer-ticket-api-74c585b587-vnh14	0/1	ImagePullBackOff	0	108s

```
(venv) PS C:\DEVOPS-PROJECT> kubectl get service
```

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
customer-ticket-service	NodePort	10.100.4.161	<none>	5000:30080/TCP	2m27s
kubernetes	ClusterIP	10.96.0.1	<none>	443/TCP	6d23h

```
o (venv) PS C:\DEVOPS-PROJECT>
```

kubectl get pods: ImagePullBackOff

Docker not running

Start Docker Desktop, wait for "Engine running", then minikube start.

API refused

Cluster was down. Start minikube first, then kubectl apply.

ImagePullBackOff

Image not available to the cluster: load/build it inside Minikube; check image name and pull policy.

Key Takeaways

SOURCE
Git + GitHub

CI/CD
Jenkins

CONTAINER
Docker

ORCHESTRATION
Kubernetes

- Connected four tools around one real application, from code push to deployed workload
- Debugged Git, Jenkins, Docker and Kubernetes failures instead of only showing the happy path
- Verified every stage: checkout, build, rollout and the live endpoint

github.com/bhattraul76660-cyber/cloud-support-devops-project